

GUÍA DE LA ENTREGA 3 — TADs lineales y excepciones

Vence: domingo **04-oct-2026, 23:59** (hora Argentina).

Vale: **12 %** de la nota final (la entrega más pesada).

Tag en GitHub: exactamente **entrega-3**.

Esta guía tiene **teoría + práctica** para que puedas entender cada concepto desde cero.
Léela entera antes de escribir una línea de código.

0. QUÉ SE CORRIGE ESTA VEZ (checklist rápido)

Son 8 ítems de la rúbrica. Si hacés solo esto, aprobás la E3:

#	Qué se corrige	Peso interno	En palabras simples
3.1	ListaEnlazada propia con nodos	20 %	Tus listas NO pueden usar list de Python por abajo
3.2	Iterador funcionando; el listado lo usa	10 %	__iter__ / __next__ para recorrer la lista
3.3	Pila y Cola sobre la lista, usadas en el dominio	20 %	Historial (deshacer) y cola de turnos
3.4	Colección principal sobre ListaEnlazada , con tope	15 %	Equipo de 6, menú semanal o playlist con límite
3.5	Encapsulamiento	10 %	El menú no accede a atributos internos directamente
3.6	Excepciones propias capturadas en el menú	10 %	Nunca except: pelado
3.7	Protocolo ejecutado	10 %	Tabla con columna pasa / no pasa
3.8	Tag corre; declaración de IA al día	5 %	entrega-3 funciona; DECLARACION_IA.md actualizado

Regla de oro: lo que andaba en E1 y E2 tiene que seguir andando. Si rompés algo viejo, pierdes puntos del ítem nuevo.

1. TEORÍA: QUÉ ES UNA LISTA ENLAZADA

1.1 El problema que resuelve

Imaginate que tenés una lista de contactos. En Python, usás una **list** normal:

```
contactos = ["Ana", "Bruno", "Carla"]
```

Esa **list** de Python vive en memoria **contigua**: todos los datos juntos, uno después del otro. Si querés agregar algo en el medio, Python tiene que **mover todos los elementos** para hacer lugar. Eso es lento cuando la lista es grande.

Una **lista enlazada** resuelve esto distinto: cada dato vive en un **nodo**, y cada nodo tiene una **flecha** (un enlace) que apunta al siguiente nodo. No necesitan estar juntos en memoria.

```
[Cara] → [Nariz] → [Boca] → [Oreja] → None
```

Cada nodo tiene:

- **dato**: la información (ej: el nombre de un Pokémon)
- **siguiente**: la referencia al nodo que viene después (o **None** si es el último)

1.2 Analogía del mundo real

Pensá en una **cadena de mensajes encadenados**: cada persona tiene un mensaje y el nombre de la siguiente persona a quien pasárselo. Si alguien quiere agregar un mensaje en el medio, solo tiene que cambiar la flecha de un nodo, no mover toda la cadena.

1.3 Nodo: el bloque básico

```
class Nodo:
    """Un eslabón de la cadena. Contiene un dato y la referencia al
    siguiente."""
    def __init__(self, dato, siguiente=None):
        self.dato = dato
        self.siguiente = siguiente
```

¿Qué hace cada cosa?

- **self.dato = dato**: guarda el dato (un string, un número, un objeto...)
- **self.siguiente = siguiente**: apunta al siguiente nodo. Si es **None**, es el último.
- **siguiente=None**: por defecto, el nodo es el último de la cadena.

Crear una cadena de 3 nodos:

```
nodo3 = Nodo("Carla")           # último nodo, siguiente = None
nodo2 = Nodo("Bruno", nodo3)    # apunta a nodo3
```

```
nodo1 = Nodo("Ana", nodo2)      # apunta a nodo2
# Cadena: Ana → Bruno → Carla → None
```

1.4 ListaEnlazada: la estructura completa

La lista enlazada tiene un **atributo** `_cabeza` (la "cabeza" o primer nodo) y un **atributo** `_tamanio` para saber cuántos nodos hay.

```
class ListaEnlazada:
    def __init__(self):
        self._cabeza = None    # no hay nodos todavía
        self._tamanio = 0      # tamaño inicial: 0
```

Operaciones principales

`esta_vacia()` — ¿Hay al menos un nodo?

```
def esta_vacia(self):
    return self._cabeza is None
```

`tamanio()` — ¿Cuántos nodos hay?

```
def tamanio(self):
    return self._tamanio
```

`insertar_al_inicio(dato)` — Agregar al principio (es la operación más rápida de una lista enlazada)

Pasos:

1. Crear un nuevo nodo con el dato.
2. Ese nodo nuevo apunta al nodo que era la cabeza.
3. La cabeza ahora es el nodo nuevo.
4. Sumar 1 al tamaño.

```
def insertar_al_inicio(self, dato):
    nuevo = Nodo(dato, self._cabeza)
    self._cabeza = nuevo
    self._tamanio += 1
```

Visualización:

Antes: Bruno → Carla → None
 ^ cabeza

nuevo = Nodo("Ana", Bruno)

Después: Ana → Bruno → Carla → None
 ^ cabeza

`insertar_al_final(dato)` — Agregar al final

Pasos:

1. Crear un nuevo nodo.
2. Recorrer la lista hasta el último nodo.
3. El último nodo ahora apunta al nuevo.
4. Sumar 1 al tamaño.

```
def insertar_al_final(self, dato):
    nuevo = Nodo(dato)
    if self.esta_vacia():
        self._cabeza = nuevo
    else:
        actual = self._cabeza
        while actual.siguiente is not None:
            actual = actual.siguiente
        actual.siguiente = nuevo
    self._tamanio += 1
```

`buscar(dato)` — Buscar un dato en la lista

```
def buscar(self, dato):
    actual = self._cabeza
    while actual is not None:
        if actual.dato == dato:
            return actual          # lo encontró
        actual = actual.siguiente
    return None                   # no lo encontró
```

`eliminar(dato)` — Sacar un nodo de la lista

Tres casos:

1. **Es el primero:** la cabeza apunta al segundo nodo.
2. **Es del medio:** el nodo anterior apunta al que viene después del que se saca.
3. **No existe:** no hacer nada.

```
def eliminar(self, dato):
    if self.esta_vacia():
        return
    # Caso 1: es la cabeza
    if self._cabeza.dato == dato:
        self._cabeza = self._cabeza.siguiente
        self._tamanio -= 1
        return
    # Caso 2: buscar el nodo anterior
    actual = self._cabeza
    while actual.siguiente is not None:
        if actual.siguiente.dato == dato:
            actual.siguiente = actual.siguiente.siguiente
            self._tamanio -= 1
            return
        actual = actual.siguiente
```

1.5 ¿Por qué NO usar `list` de Python?

La consigna dice textualmente: "un `list` de Python por debajo cuenta como N ". Esto significa que si hacés:

```
class ListaEnlazada:
    def __init__(self):
        self._datos = [] # ¡INCORRECTO! Usa list de Python por abajo
```

Se considera N (no cumple). La lista enlazada tiene que estar hecha con nodos y referencias (`Nodo + siguiente`), no con una `list`.

2. TEORÍA: QUÉ ES UN ITERADOR

2.1 El problema que resuelve

Si querés **recorrer** todos los nodos de tu lista enlazada para mostrarlos en pantalla, tenés que hacer un `while` manual cada vez:

```
actual = lista._cabeza
while actual is not None:
    print(actual.dato)
    actual = actual.siguiente
```

Un **iterador** permite hacer `for elemento in lista_enlazada` y que Python se encargue de recorrer automáticamente. Es como tener un pasajero que se sienta al inicio de la lista y se va moviendo de nodo en nodo hasta el final.

2.2 `__iter__` y `__next__` explicados

Python llama a `__iter__` cuando empieza un `for`. `__iter__` devuelve un **objeto iterador**. Ese objeto tiene `__next__`, que devuelve el **siguiente elemento** cada vez que se llama.

Cuando no queda más, `__next__` lanza `StopIteration` (Python lo maneja solo, no tenés que preocuparte).

```
class ListaEnlazada:
    # ... (tus métodos anteriores)

    def __iter__(self):
        """Python llama a esto cuando empezás un 'for'. Devuelve el
        iterador."""
        return self._Iterador(self._cabeza)

    class _Iterador:
        """Clase interna que hace el recorrido nodo a nodo."""
        def __init__(self, cabeza):
            self._actual = cabeza

        def __iter__(self):
            return self

        def __next__(self):
            if self._actual is None:
                raise StopIteration      # fin de la lista
            dato = self._actual.dato
            self._actual = self._actual.siguiente
            return dato
```

Uso:

```
lista = ListaEnlazada()
lista.insertar_al_final("Ana")
lista.insertar_al_final("Bruno")
lista.insertar_al_final("Carla")

for nombre in lista:
    print(nombre)
# Salida:
# Ana
# Bruno
# Carla
```

También podés usar `yield` (es más corto y Python lo entiende igual):

```
def __iter__(self):
    actual = self._cabeza
    while actual is not None:
        yield actual.dato
        actual = actual.siguiente
```

`yield` convierte la función en un generador: cada vez que Python la llama, devuelve un dato y **pausa** hasta la próxima llamada. Cuando no queda más, lanza `StopIteration` solo.

2.3 Por qué importa para la E3

La rúbrica pide que el **listado del catálogo o de la colección** use el iterador. En tu `main.py`, cuando el usuario elige "listar catálogo", tu código debe tener:

```
# CORRECTO – usa el iterador
for item in catalogo:
    print(item)

# INCORRECTO – accede a atributos internos
actual = catalogo._cabeza
while actual is not None:
    print(actual.dato)
    actual = actual.siguiente
```

3. TEORÍA: QUÉ ES UNA PILA (STACK)

3.1 Definición

Una **pila** es una colección donde los elementos se añaden y se sacan **por el mismo extremo** (la **tapa** o **tope**). Funciona **LIFO**: **Last In, First Out** (el último en entrar es el primero en salir).

3.2 Analogía del mundo real

Pensá en una pila de platos:

- Apilas un plato nuevo **arriba** → `apilar()`
- Sacás el plato de **arriba** → `desapilar()`
- Mirás cuál es el plato de arriba sin sacarlo → `ver_tope()`

O pensá en el **deshacer (Ctrl+Z)** de un editor: cada acción se guarda en una pila. Cuando querés deshacer, tomás la última acción hecha.

3.3 Operaciones

Método	Qué hace	Complejidad
--------	----------	-------------

Método	Qué hace	Complejidad
<code>apilar(dato)</code>	Agrega un elemento al tope	O(1)
<code>desapilar()</code>	Saca y devuelve el del tope. Si está vacía, lanza error	O(1)
<code>ver_tope()</code>	Mira el del tope sin sacarlo	O(1)
<code>esta_vacia()</code>	¿Hay elementos?	O(1)

3.4 Implementación sobre `ListaEnlazada`

Como las inserciones y eliminaciones al inicio de una lista enlazada son **O(1)**, la pila se implementa **insertando y sacando siempre por la cabeza**:

```
class Pila:
    """Pila implementada sobre ListaEnlazada (LIFO)."""
    def __init__(self):
        self._items = ListaEnlazada()

    def apilar(self, dato):
        """Agrega al tope. Equivale a insertar_al_inicio en la lista."""
        self._items.insertar_al_inicio(dato)

    def desapilar(self):
        """Sacá el tope. Si la pila está vacía, lanzá PilaVacíaError."""
        if self.esta_vacia():
            raise PilaVacíaError("No hay elementos en el historial para deshacer.")
        tope = self._items.buscar(self._items._cabeza.dato)
        self._items.eliminar(self._items._cabeza.dato)
        return tope.dato

    def ver_tope(self):
        """Mirá el del tope sin sacarlo."""
        if self.esta_vacia():
            raise PilaVacíaError("La pila está vacía.")
        return self._items._cabeza.dato

    def esta_vacia(self):
        return self._items.esta_vacia()
```

Ojo: accedés a `_cabeza` solo dentro de la clase `Pila`, no desde afuera. Esto es encapsulamiento (ver sección 5).

3.5 Uso en el dominio

Tema	Uso de la pila
Pokédex	Historial de acciones (deshacer: volvés al Pokémon anterior visitado)

Tema	Uso de la pila
Recetario	Historial de recetas cocinadas
Música	Historial de reproducción (deshacer última canción escuchada)

```
# Ejemplo: historial de Pokédex
historial = Pila()
historial.apilar("Pikachu")      # visitaste a Pikachu
historial.apilar("Charmander")  # visitaste a Charmander
historial.desapilar()           # volvés a Pikachu (deshacer)
```

4. TEORÍA: QUÉ ES UNA COLA (QUEUE)

4.1 Definición

Una **cola** es una colección donde se agregan por un extremo (**final** o **rear**) y se sacan por el otro (**frente** o **front**). Funciona **FIFO**: **F**irst **I**n, **F**irst **O**ut (el primero en entrar es el primero en salir).

4.2 Analogía del mundo real

Pensá en una fila en el banco:

- Llegás y te parás al **final** de la fila → `encolar()`
- Te atienden y te vas del **frente** → `desencolar()`
- Mirás quién es el próximo sin irte → `ver_frente()`

4.3 Operaciones

Método	Qué hace	Complejidad
<code>encolar(dato)</code>	Agrega al final	O(1) si tenés cola final, O(n) si solo tenés cabeza
<code>desencolar()</code>	Saca y devuelve del frente	O(1)
<code>ver_frente()</code>	Mira el del frente sin sacarlo	O(1)
<code>esta_vacia()</code>	¿Hay elementos?	O(1)

4.4 Implementación sobre `ListaEnlazada`

La cola necesita eficiencia en ambos extremos. La forma más simple: insertar al final y sacar desde la cabeza.

```
class Cola:
    """Cola implementada sobre ListaEnlazada (FIFO)."""
    def __init__(self):
        self._items = ListaEnlazada()
```

```

def encolar(self, dato):
    """Agrega al final de la cola."""
    self._items.insertar_al_final(dato)

def desencolar(self):
    """Sacá del frente. Si la cola está vacía, lanzá ColaVacíaError."""
    if self.esta_vacia():
        raise ColaVacíaError("No hay elementos en la cola.")
    frente = self._items._cabeza.dato
    self._items.eliminar(frente)
    return frente

def ver_frente(self):
    """Mirá el del frente sin sacarlo."""
    if self.esta_vacia():
        raise ColaVacíaError("La cola está vacía.")
    return self._items._cabeza.dato

def esta_vacia(self):
    return self._items.esta_vacia()

```

4.5 Uso en el dominio

Tema	Uso de la cola
Pokédex	Cola de turnos de combate
Recetario	Cola de preparación (primer receta ingresada, primero en cocinarse)
Música	Cola de reproducción (primer tema encolado, primero en sonar)

```

# Ejemplo: cola de preparación en recetario
cola = Cola()
cola.encolar("Sofrito")
cola.encolar("Masa de empanadas")
cola.encolar("Relleno")
cola.desencolar() # sale "Sofrito" (primero en entrar, primero en salir)

```

5. TEORÍA: ENCAPSULAMIENTO

5.1 Qué significa

Encapsulamiento significa que los datos internos de una clase (atributos) no deben ser accedidos directamente desde afuera. Se accede solo a través de **métodos** (funciones de la clase).

5.2 Ejemplo

```
# CORRECTO – encapsulado: se usa self._atributo (guión bajo) y métodos públicos
class Pila:
    def __init__(self):
        self._items = ListaEnlazada()    # _items = atributo interno

    def apilar(self, dato):                # método público
        self._items.insertar_al_inicio(dato)

    def desapilar(self):                  # método público
        if self.esta_vacia():
            raise PilaVacíaError()
        tope = self._items._cabeza.dato
        self._items.eliminar(tope)
        return tope

# INCORRECTO – el menú accede directamente a atributos internos
# esto viola el encapsulamiento (rúbrica 3.5)
historial = Pila()
historial._items._cabeza.dato = "algo"  # ¡NO HACER ESTO!
```

Regla: desde `main.py` (el menú), solo llamás métodos como `historial.apilar()`, `historial.desapilar()`, `historial.esta_vacia()`. **Nunca** accedés a `historial._items._cabeza` ni a ningún otro atributo con guion bajo.

5.3 Por qué importa

El evaluador puede leer tu código y ver si el menú (en `main.py`) intenta manipular directamente atributos internos de las clases TAD. Si lo hace, pone **N** en el ítem 3.5.

6. TEORÍA: EXCEPCIONES PROPIAS

6.1 Qué son

Las **excepciones** son errores que tu programa puede lanzar cuando algo inesperado ocurre. Una **excepción propia** es una que vos creás, específica para tu dominio.

6.2 El archivo `src/excepciones.py` ya está armado

Revisá que tenga estas clases (ya están en el esqueleto):

```
class ItemNoEncontradoError(Exception):
    pass

class ColeccionLlenaError(Exception):
    pass

class ColeccionVacíaError(Exception):
    pass
```

```
class PilaVacíaError(Exception):
    pass

class ColaVacíaError(Exception):
    pass

class ArchivoInvalidoError(Exception):
    pass
```

6.3 Cómo lanzar excepciones propias

```
from src.excepciones import PilaVacíaError, ColecciónLlenaError

class Pila:
    def desapilar(self):
        if self.esta_vacia():
            raise PilaVacíaError("No hay elementos para deshacer.")
        # ... lógica normal
```

6.4 Cómo capturar en el menú (NUNCA `except:` pelado)

INCORRECTO (`except` pelado):

```
try:
    item = coleccion.desencolar()
except:
    # ¡Esto atrapa CUALQUIER error, incluso los de Python!
    print("Error")
```

CORRECTO (`except` con excepción específica):

```
try:
    item = coleccion.desencolar()
except ColaVacíaError as e:
    print(f"No se puede desenchufar: {e}")
except ColecciónVacíaError as e:
    print(f"La colección está vacía: {e}")
```

6.5 Excepciones que tenés que capturar en el menú

Cuándo	Qué excepción capturar	Qué mensaje mostrar
Colección Llena (equipo de 6 lleno)	<code>ColecciónLlenaError</code>	"El equipo está lleno (máximo 6). No se puede agregar más."

Cuándo	Qué excepción capturar	Qué mensaje mostrar
Pila vacía (quiere deshacer sin historial)	<code>PilaVacíaError</code>	"No hay acciones en el historial para deshacer."
Cola vacía (quiere atender sin turnos)	<code>ColaVacíaError</code>	"No hay elementos en la cola."
Ítem no encontrado	<code>ItemNoEncontradoError</code>	"No se encontró ese elemento."

7. TEORÍA: LA COLECCIÓN PRINCIPAL CON TOPE

7.1 Qué es

La colección principal es el **equipo de 6 Pokémon**, el **menú de la semana** o la **playlist**. Tiene un **tope** (límite máximo).

Por ejemplo, un equipo de Pokémon tiene máximo **6** pokémon. Si intentás agregar un séptimo, debe fallar con una excepción propia.

7.2 Implementación

```
class Equipo:
    def __init__(self, tope=6):
        self._pokemones = ListaEnlazada()
        self._tope = tope

    def agregar(self, pokemon):
        if self._pokemones.tamano() >= self._tope:
            raise ColeccionLlenaError(f"El equipo está lleno (máximo {self._tope}).")
        self._pokemones.insertar_al_final(pokemon)

    def eliminar(self, pokemon):
        self._pokemones.eliminar(pokemon)

    def listar(self):
        for p in self._pokemones:
            print(p)
```

Regla clave: `self._pokemones` es una `ListaEnlazada` (no una `list` de Python). El tope se verifica con `self._pokemones.tamano()`.

8. EJEMPLO COMPLETO INTEGRADO

Este es el corazón de la E3. Acá tenés un ejemplo funcional de todos los TADs juntos:

```

from src.tads.nodo import Nodo
from src.tads.lista_enlazada import ListaEnlazada
from src.tads.pila import Pila
from src.tads.cola import Cola
from src.excepciones import PilaVacíaError, ColaVacíaError, ColecciónLlenaError

# --- ListaEnlazada con iterador (archivo: src/tads/lista_enlazada.py) ---
class ListaEnlazada:
    def __init__(self):
        self._cabeza = None
        self._tamaño = 0

    def esta_vacia(self):
        return self._cabeza is None

    def tamaño(self):
        return self._tamaño

    def insertar_al_inicio(self, dato):
        self._cabeza = Nodo(dato, self._cabeza)
        self._tamaño += 1

    def insertar_al_final(self, dato):
        nuevo = Nodo(dato)
        if self.esta_vacia():
            self._cabeza = nuevo
        else:
            actual = self._cabeza
            while actual.siguiente is not None:
                actual = actual.siguiente
            actual.siguiente = nuevo
        self._tamaño += 1

    def eliminar(self, dato):
        if self.esta_vacia():
            return
        if self._cabeza.dato == dato:
            self._cabeza = self._cabeza.siguiente
            self._tamaño -= 1
            return
        actual = self._cabeza
        while actual.siguiente is not None:
            if actual.siguiente.dato == dato:
                actual.siguiente = actual.siguiente.siguiente
                self._tamaño -= 1
                return
            actual = actual.siguiente

    def buscar(self, dato):
        actual = self._cabeza
        while actual is not None:
            if actual.dato == dato:

```

```

        return actual
        actual = actual.siguiente
    return None

def __iter__(self):
    actual = self._cabeza
    while actual is not None:
        yield actual.dato
        actual = actual.siguiente

# --- Pila (archivo: src/tads/pila.py) ---
class Pila:
    def __init__(self):
        self._items = ListaEnlazada()

    def apilar(self, dato):
        self._items.insertar_al_inicio(dato)

    def desapilar(self):
        if self.esta_vacia():
            raise PilaVacíaError("No hay elementos para deshacer.")
        tope = self.ver_tope()
        self._items.eliminar(tope)
        return tope

    def ver_tope(self):
        if self.esta_vacia():
            raise PilaVacíaError("La pila está vacía.")
        return self._items._cabeza.dato

    def esta_vacia(self):
        return self._items.esta_vacia()

# --- Cola (archivo: src/tads/cola.py) ---
class Cola:
    def __init__(self):
        self._items = ListaEnlazada()

    def encolar(self, dato):
        self._items.insertar_al_final(dato)

    def desencolar(self):
        if self.esta_vacia():
            raise ColaVacíaError("No hay elementos en la cola.")
        frente = self.ver_frente()
        self._items.eliminar(frente)
        return frente

    def ver_frente(self):
        if self.esta_vacia():
            raise ColaVacíaError("La cola está vacía.")

```

```

        return self._items._cabeza.dato

    def esta_vacia(self):
        return self._items.esta_vacia()

# --- Equipo con tope (archivo: src/dominio/equipo.py o menu_semanal.py) ---
class Equipo:
    def __init__(self, tope=6):
        self._pokemones = ListaEnlazada()
        self._tope = tope

    def agregar(self, pokemon):
        if self._pokemones.tamano() >= self._tope:
            raise ColeccionLlenaError(f"El equipo está lleno (máximo {self._tope}).")
        self._pokemones.insertar_al_final(pokemon)

    def eliminar(self, pokemon):
        self._pokemones.eliminar(pokemon)

    def listar(self):
        for p in self._pokemones:
            print(f" {p}")

```

9. EJEMPLO DEL MENÚ CON EXCEPCIONES CAPTURADAS

Acá tenés cómo debería verse la sección de menú en tu `main.py` (o donde manejes las opciones 6, 7 y 8):

```

from src.excepciones import ColeccionLlenaError, PilaVacíaError, ColaVacíaError

def menu_coleccion(equipo, pila_historial, cola_turnos):
    while True:
        print("\n--- Colección Principal ---")
        print("1. Agregar al equipo")
        print("2. Listar equipo")
        print("3. Deshacer (pila)")
        print("4. Siguiendo turno (cola)")
        print("0. Volver")

        opcion = input("> ").strip()

        if opcion == "0":
            break
        elif opcion == "1":
            nombre = input("Nombre del elemento: ").strip()
            try:
                equipo.agregar(nombre)
                print(f"✅ Agregado: {nombre}")

```

```

except ColeccionLlenaError as e:
    print(f"❌ {e}")
elif opcion == "2":
    equipo.listar()
elif opcion == "3":
    try:
        item = pila_historial.desapilar()
        print(f"🗑 Deshecho: {item}")
    except PilaVacuaError as e:
        print(f"❌ {e}")
elif opcion == "4":
    try:
        item = cola_turnos.desencolar()
        print(f"➡ Turno atendido: {item}")
    except ColaVacuaError as e:
        print(f"❌ {e}")

```

Nota: no hay ningún **except**: sin especificar. Cada **except** menciona la excepción exacta.

10. PROTOCOLO DE PRUEBAS (SECCIÓN 3.7 DE LA RÚBRICA)

En `docs/PROTOCOLO_PRUEBAS.md` tenés que **ejecutar** los casos de E3 y completar la columna "Resultado" con **pasa** o **no pasa**.

Los casos de E3 son (ver el protocolo del esqueleto, IDs P05–P08):

ID	Acción	Resultado esperado
P05	Agregar a la colección hasta el tope	el séptimo falla con excepción propia
P06	Desapilar historial vacío	excepción propia, menú sigue
P07	Desencolar cola vacía	excepción propia, menú sigue
P08	Listar colección con iterador	el orden coincide con las inserciones

¡Ojo! Esto es nuevo en E3: la columna debe estar **ejecutada** (con **pasa** o **no pasa**), no solo escrita como "no corrido" como en E2.

11. ESTRUCTURA DEL REPO PARA E3

Tu repo debe tener esta estructura. Los archivos **marcados con 1 son nuevos o modificados para E3**:

```

ayed-2026-recetario/
├── README.md           # actualizar con info de E3
├── src/
│   ├── config.py
│   ├── main.py        # completar opciones 6, 7, 8 del menú
│   └── excepciones.py # ya está, revisar que esté bien

```

```

|   ├── dominio/
|   |   ├── receta.py           # actualizar: catálogo sobre ListaEnlazada
|   |   ├── libro_recetas.py    # catálogo sobre ListaEnlazada
|   |   ├── menu_semanal.py     # ↑ colección con tope
|   |   ├── historial.py        # ↑ Pila
|   |   ├── cola_preparacion.py # ↑ Cola
|   |   └── desglose.py         # recursión de E2 intacta
|   ├── persistencia/
|   |   └── texto.py
|   └── tads/                    # ↑ todos los archivos TAD implementados
|       ├── nodo.py             # ya tiene el Nodo, verificar
|       ├── lista_enlazada.py    # ↑ implementación completa
|       ├── pila.py              # ↑ implementación completa
|       └── cola.py              # ↑ implementación completa
|   ├── data/
|   ├── docs/
|   ├── INFORME.md              # + §4 TADs (operaciones e invariante)
|   ├── PROTOCOLO_PRUEBAS.md    # ↑ ejecutado (pasa / no pasa)
|   └── DECLARACION_IA.md       # ↑ fila E3

```

12. PASO A PASO PARA ENTREGAR

12.1 Implementá los TADs

1. Completá `src/tads/lista_enlazada.py` con todos los métodos + `__iter__`
2. Completá `src/tads/pila.py` usando `ListaEnlazada`
3. Completá `src/tads/cola.py` usando `ListaEnlazada`
4. Verificá que `python -m src.main` arranca sin errores

12.2 Implementá la colección del dominio

5. Creá (o completá) el archivo de colección principal (equipo/menú/playlist) que use `ListaEnlazada`
6. Implementá el tope (máximo de elementos)
7. Integrá la pila y la cola en tu menú

12.3 Integrá las excepciones en el menú

8. Capturá `ColeccionLlenaError`, `PilaVacíaError`, `ColaVacíaError` en las opciones 6, 7 y 8 del menú
9. Verificá que no hay ningún `except:` pelado

12.4 Ejecutá el protocolo de pruebas

10. Corré cada caso P05–P08 y poné `pasa` o `no pasa` en `docs/PROTOCOLO_PRUEBAS.md`

12.5 Actualizá la documentación

11. `docs/INFORME.md` → sección 4: TADs con tabla de operaciones e invariante
12. `docs/DECLARACION_IA.md` → fila E3

12.6 Taggeá y pusheá

```
git add .
git status                # revisá que esté todo lo que querés subir
git commit -m "E3: lista enlazada, pila y cola"
git tag entrega-3
git push
git push origin entrega-3
```

12.7 Mandá el mail

Destinatarios: diego.ambrossio@unab.edu.ar y angel.bianco@unab.edu.ar (en el mismo correo)

Asunto: AyED C2 2026 Comision X - GRUPO XX - Entrega 3

Cuerpo:

Repo: <https://github.com/usuario/ayed-2026-recetario>
Entrega de E3: ListaEnlazada propia con iterador, Pila y Cola sobre esa lista, menú con tope, excepciones propias y protocolo ejecutado. Tag: entrega-3.

13. CHECKLIST FINAL ANTES DEL DOMINGO 04-oct 23:59

- ☐ `ListaEnlazada` implementada con nodos (¡no `list` de Python por debajo!)
- ☐ `__iter__` / `__next__` (o `yield`) funcionando
- ☐ El listado del catálogo usa el iterador (`for item in lista`)
- ☐ `Pila` implementada sobre `ListaEnlazada` (apilar/desapilar/ver_tope/esta_vacia)
- ☐ `Cola` implementada sobre `ListaEnlazada` (encolar/desencolar/ver_frente/esta_vacia)
- ☐ Colección principal (equipo/menú/playlist) usa `ListaEnlazada` con tope
- ☐ Excepciones propias lanzadas en la colección llena, pila vacía, cola vacía
- ☐ Excepciones capturadas en el menú **sin** `except`: pelado
- ☐ Encapsulamiento: el menú no accede a `_cabeza`, `_items`, etc. directamente
- ☐ `docs/PROTOCOLO_PRUEBAS.md` con casos P05–P08 ejecutados (pasa / no pasa)
- ☐ `docs/INFORME.md` §4: tabla de TADs con operaciones e invariante
- ☐ `docs/DECLARACION_IA.md` con fila E3
- ☐ Lo de E1 (listar catálogo) y E2 (recursión) sigue andando
- ☐ `python -m src.main` arranca sin error
- ☐ `git tag entrega-3` hecho, pusheado, y también `git push origin entrega-3`
- ☐ Mail a Ambrossio y Bianco con asunto "...Entrega 3", repo y 1 línea de qué incluye

14. ERRORES QUE CUESTAN PUNTOS (y cómo evitarlos)

Error	Consecuencia	Cómo evitarlo
-------	--------------	---------------

Error	Consecuencia	Cómo evitarlo
Usar <code>list</code> de Python como base	N en ítem 3.1	Hacer la lista con <code>Nodo</code> + <code>siguiente</code>
Sin iterador en el listado	N en ítem 3.2	Implementar <code>__iter__</code> / <code>__next__</code> o <code>yield</code>
<code>Pila</code> o <code>Cola</code> que usen <code>list</code>	N en ítem 3.3	Deben usar <code>ListaEnlazada</code>
Menú accede a <code>._cabeza</code>	N en ítem 3.5	Encapsular; solo llamar métodos
<code>except:</code> pelado	N en ítem 3.6	Siempre especificar la excepción
Protocolo no ejecutado	N en ítem 3.7	Correr cada caso y poner pasa/no pasa
Tag mal escrito	No se encuentra	Exactamente <code>entrega-3</code>
Romper E1 o E2	N en regresión	Probar antes de taguear
Entregar después del domingo 23:59	-25 % si llega lunes/martes	Terminar antes

15. GLOSAARIO RÁPIDO

Término	Significado
Nodo	Un bloque de datos + referencia al siguiente
Lista enlazada	Cadena de nodos conectados por referencias
Iterador	Objeto que permite recorrer elementos uno a uno con <code>for</code>
<code>__iter__</code>	Método que devuelve el iterador (llamado por <code>for</code>)
<code>__next__</code>	Método que devuelve el siguiente elemento (llamado por <code>for</code>)
<code>yield</code>	Genera un valor y pausa; la próxima vez continúa desde donde se quedó
Pila (Stack)	LIFO: último en entrar, primero en salir (tipo apilar platos)
Cola (Queue)	FIFO: primero en entrar, primero en salir (tipo fila de banco)
Tope	Elemento arriba de la pila o límite máximo de la colección
Encapsulamiento	Los datos internos no se acceden directamente desde afuera
Excepción propia	Clase de error que vos creás para tu dominio
<code>raise</code>	Lanza una excepción
<code>try/except</code>	Intentá algo; si falla con la excepción indicada, maneja
<code>StopIteration</code>	Señal interna de Python que avisa que terminó la iteración